

ESP32 + MPU6050 무선 IMU 통신 Codex 자동화 실행 가이드

Windows / ESP-IDF / Wi-Fi UDP / VSCode Python Receiver

항목	설정
ESP-IDF 프로젝트	C:\Users\Wkdg57\esp\Wesp32_test
ESP32 소스	C:\Users\Wkdg57\esp\Wesp32_test\main\Wesp32_test.c
ESP32 포트	COM3 (Silicon Labs CP210x USB to UART Bridge)
핫스팟 SSID	Dofbot
핫스팟 비밀번호	qweasdzxc1!
UDP 포트	5005
노트북 수신기	imu_receiver.py

목표: ESP32 와 노트북 사이의 USB 시리얼 통신을 사용하지 않고, ESP32 가 Dofbot 핫스팟에 접속하여 MPU6050 의 ROLL / CTRL_ROLL / PITCH / STATE 값을 UDP 로 노트북에 무선 전송한다. USB 는 펌웨어 플래시 및 디버깅 시에만 사용한다.

1. Codex 가 최종적으로 만들어야 하는 상태

```
MPU6050
| I2C (GPIO21 SDA / GPIO22 SCL)
v
ESP32
| Wi-Fi UDP
v
Dofbot Hotspot
| Wi-Fi
v
Windows Laptop / VSCode
|
+--> imu_receiver.py
      ROLL / CTRL_ROLL / PITCH / STATE 출력
```

- 실사용 중에는 ESP32-노트북 USB 케이블이 없어도 통신되어야 한다.
- ESP32 는 별도 전원(배터리/승압모듈)으로 켜져 있어야 한다.
- 노트북과 ESP32 는 모두 동일한 Dofbot 핫스팟에 연결되어 있어야 한다.
- 노트북 수신기는 UDP 5005 포트에서 데이터를 기다린다.
- ESP32 가 보내는 문자열 형식은 기존 idf.py monitor 출력과 동일하게 유지한다.

```
ROLL: 10.49 | CTRL_ROLL: -10.49 | PITCH: -12.43 | STATE:STOP
```

2. Codex 에 그대로 붙여넣을 작업 지시 프롬프트

아래 블록 전체를 Codex 작업 지시로 전달한다. Codex 가 현재 저장소를 먼저 확인하고, 필요한 파일만 수정하도록 설계한 프롬프트다.

너는 Windows 환경의 ESP-IDF 프로젝트를 직접 점검하고 수정하는 개발 에이전트다.

목표:

ESP32 + MPU6050 장갑 센서가 Dofbot Wi-Fi 핫스팟을 통해 Windows 노트북과 UDP 무선 통신하도록 구성하라. ESP32 와 노트북 사이의 USB 연결은 펌웨어 flash/debug 용도일 뿐이며, 최종 수신 테스트는 USB 없이 동작해야 한다.

현재 환경:

- 프로젝트 루트: C:\Users\kdg57\esp\esp32_test
- ESP32 소스: C:\Users\kdg57\esp\esp32_test\main\esp32_test.c
- component CMake: C:\Users\kdg57\esp\esp32_test\main\CMakeLists.txt
- ESP32 USB-UART: COM3 / Silicon Labs CP210x
- ESP-IDF: C:\esp\v6.1-beta1\esp-idf
- Wi-Fi SSID: Dofbot
- Wi-Fi password: qweasdzxc1!
- UDP port: 5005
- MPU6050 I2C: SDA GPIO21, SCL GPIO22, address 0x68

해야 할 일:

1. 프로젝트 전체를 먼저 읽고 현재 esp32_test.c의 MPU6050 측정/필터/방향판정 로직을 보존한다. 기존 정상 동작하던 ROLL, CTRL_ROLL, PITCH, STATE 계산을 함부로 재작성하지 마라.
2. esp32_test.c에 Wi-Fi Station 연결 로직이 있는지 확인하고, 없다면 ESP-IDF 방식으로 추가한다.
3. Dofbot에 접속 후 ESP32가 받은 IP를 로그로 확인할 수 있도록 한다.
4. UDP socket을 생성하고 ROLL / CTRL_ROLL / PITCH / STATE를 5005번 포트로 전송한다.
5. 우선 프로젝트에 이미 설정된 UDP_TARGET_IP가 있으면 유지한다. 255.255.255.255 broadcast가 모바일 핫스팟에서 막힐 경우를 대비해 Windows 노트북의 Dofbot Wi-Fi IPv4 주소를 자동 탐지하여 unicast로 바꿀 수 있는 보조 스크립트를 만든다.
6. C:\Users\kdg57\imu_receiver 폴더를 만들고 imu_receiver.py를 생성한다.
7. imu_receiver.py는 0.0.0.0:5005에 bind하고 UTF-8 문자열을 받아 timestamp, ESP32 source IP, message를 출력한다.
8. Windows 방화벽 때문에 UDP 5005가 막힐 수 있으므로 현재 규칙을 점검하고, 관리자 권한이 있으면 "ESP32 IMU UDP 5005" 인바운드 UDP 규칙을 생성하는 PowerShell 스크립트를 제공한다. 기존 규칙을 불필요하게 삭제하지 마라.
9. setup_imu_receiver.ps1을 생성해 다음을 자동화한다:
 - Python 설치 여부 확인
 - C:\Users\kdg57\imu_receiver 생성
 - imu_receiver.py 존재 확인
 - 현재 Wi-Fi SSID 확인
 - Dofbot이 아니면 경고
 - Dofbot Wi-Fi IPv4 주소 출력
 - UDP 5005 receiver 실행
10. patch_esp32_target_ip.ps1을 생성해 Dofbot Wi-Fi 어댑터의 IPv4를 읽고 esp32_test.c의 UDP_TARGET_IP define을 해당 IP로 갱신할 수 있게 한다. 수정 전 .bak 백업을 만든다.
11. main/CMakeLists.txt가 실제 파일명 esp32_test.c를 SRCS로 사용하고 있는지 확인한다. 필요한 dependency만 유지한다.
12. build 폴더가 깨져 있으면 안전하게 프로젝트 경로인지 확인한 후 build만 삭제하고 idf.py build를 실행한다.
13. flash는 자동으로 강행하지 말고, 빌드 성공 후 정확한 명령만 출력한다:
idf.py -p COM3 -b 115200 flash
14. flash 후 유선 monitor 테스트 명령은 선택적으로 출력한다:
idf.py -p COM3 monitor
15. 최종 무선 테스트 절차를 출력한다:
 - Dofbot hotspot ON
 - laptop Dofbot 연결
 - ESP32 별도 전원 ON
 - USB 분리
 - python C:\Users\kdg57\imu_receiver\imu_receiver.py
 - 장갑을 움직여 ROLL/PITCH/STATE 수신 확인
16. 변경한 모든 파일과 변경 이유를 마지막에 요약한다.
17. 추측으로 기존 MPU6050 방향 임계값이나 축 부호를 바꾸지 마라. 기존 출력이 정상이라면 그 로직을 그대로 유지한다.
18. ESP-IDF 설치 폴더(C:\esp\v6.1-beta1\esp-idf) 자체는 수정하지 마라.
19. 작업 전 현재 파일을 읽고, 변경 후 diff를 보여주고, build 결과를 확인하라.

성공 기준:

- idf.py build 성공
- ESP32가 Dofbot에 접속 가능
- Windows에서 imu_receiver.py 실행 시 Waiting 상태 진입
- USB를 제거한 상태에서도 ESP32 별도 전원 공급 후 다음 형태의 메시지가 노트북에 반복 수신됨:
[ESP32 192.168.x.x] ROLL: ... | CTRL_ROLL: ... | PITCH: ... | STATE:...

중요:

현재 프로젝트를 먼저 검사한 뒤 수정하라. 전체 파일을 무조건 덮어쓰지 말고, 기존 정상 동작하는 MPU6050 코드와 방향 판정 코드를 최대한 보존하라.

3. Codex 가 생성할 imu_receiver.py 기준 코드

```
import socket
from datetime import datetime

UDP_IP = "0.0.0.0"
UDP_PORT = 5005

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
sock.bind((UDP_IP, UDP_PORT))

print("=" * 54)
print(" ESP32 MPU6050 Wi-Fi Receiver")
print("=" * 54)
print(f"Listening UDP port : {UDP_PORT}")
print("Waiting for ESP32...\n")

try:
    while True:
        data, addr = sock.recvfrom(1024)
        message = data.decode("utf-8", errors="ignore")
        now = datetime.now().strftime("%H:%M:%S.%f")[:-3]
        print(f"[{now}] [ESP32 {addr[0]}] {message}")
except KeyboardInterrupt:
    print("\nReceiver stopped")
finally:
    sock.close()
```

4. 자동 실행용 setup_imu_receiver.ps1 예시

```
$ErrorActionPreference = "Stop"

$ReceiverDir = "C:\Users\kdg57\imu_receiver"
$ReceiverFile = Join-Path $ReceiverDir "imu_receiver.py"
$UdpPort = 5005

Write-Host "=== ESP32 IMU Wi-Fi Receiver Setup ===" -ForegroundColor Cyan
```

```

# 1) Python 확인
$python = Get-Command python -ErrorAction SilentlyContinue
if (-not $python) {
    $python = Get-Command py -ErrorAction SilentlyContinue
}
if (-not $python) {
    throw "Python을 찾을 수 없습니다. Python 설치 후 다시 실행하세요."
}

# 2) 폴더 확인
New-Item -ItemType Directory -Path $ReceiverDir -Force | Out-Null

if (-not (Test-Path $ReceiverFile)) {
    throw "imu_receiver.py가 없습니다: $ReceiverFile"
}

# 3) 현재 Wi-Fi SSID 확인
$netsh = netsh wlan show interfaces | Out-String
$ssidLine = ($netsh -split "`n" | Where-Object { $_ -match '^s*SSID\s*:' } | Select-Object -First 1)
$currentSsid = $null
if ($ssidLine) {
    $currentSsid = ($ssidLine -split ':', 2)[1].Trim()
}

if ($currentSsid -ne "Dofbot") {
    Write-Warning "현재 Wi-Fi SSID가 Dofbot이 아닙니다. 현재: $currentSsid"
} else {
    Write-Host "Wi-Fi: Dofbot 연결 확인" -ForegroundColor Green
}

# 4) Wi-Fi IPv4 찾기
$wifiIp = Get-NetIPConfiguration |
    Where-Object { $_.NetAdapter.Status -eq 'Up' -and $_.IPv4Address -and $_.InterfaceAlias -match 'Wi-Fi|WLAN|무선' } |
    Select-Object -ExpandProperty IPv4Address |
    Select-Object -ExpandProperty IPAddress |
    Select-Object -First 1

if ($wifiIp) {
    Write-Host "Laptop Wi-Fi IPv4: $wifiIp" -ForegroundColor Green
} else {
    Write-Warning "Wi-Fi IPv4 주소를 자동으로 찾지 못했습니다. ipconfig로 확인하세요."
}

# 5) UDP receiver 실행
Write-Host "UDP $UdpPort receiver 시작" -ForegroundColor Cyan
& $python.Source $ReceiverFile

```

5. Broadcast 가 안 될 때 사용할 target IP 자동 패치

```
$ErrorActionPreference = "Stop"

$SourceFile = "C:\Users\kdg57\esp\esp32_test\main\esp32_test.c"
$BackupFile = "$SourceFile.bak"

if (-not (Test-Path $SourceFile)) {
    throw "ESP32 source not found: $SourceFile"
}

$wifiIp = Get-NetIPConfiguration |
    Where-Object { $_.NetAdapter.Status -eq 'Up' -and $_.IPv4Address -and $_.InterfaceAlias -match 'Wi-Fi|WLAN|무선' } |
    Select-Object -ExpandProperty IPv4Address |
    Select-Object -ExpandProperty IPAddress |
    Select-Object -First 1

if (-not $wifiIp) {
    throw "Dofbot 에 연결된 노트북 Wi-Fi IPv4 를 찾지 못했습니다."
}

Copy-Item $SourceFile $BackupFile -Force
$content = Get-Content $SourceFile -Raw

$pattern = '#define\s+UDP_TARGET_IP\s+"[^"]+"'
$replacement = "#define UDP_TARGET_IP    \"$wifiIp\""

if ($content -notmatch $pattern) {
    throw "UDP_TARGET_IP define 을 찾지 못했습니다. 파일을 수동 확인하세요."
}

$content = [regex]::Replace($content, $pattern, $replacement, 1)
Set-Content -Path $SourceFile -Value $content -Encoding UTF8

Write-Host "Updated UDP_TARGET_IP -> $wifiIp" -ForegroundColor Green
Write-Host "Backup -> $BackupFile"
Write-Host "Next:" -ForegroundColor Cyan
Write-Host "  cd C:\Users\kdg57\esp\esp32_test"
Write-Host "  idf.py build"
Write-Host "  idf.py -p COM3 -b 115200 flash"
```

6. Windows 방화벽 UDP 5005 허용

수신기가 Waiting 상태인데 ESP32 데이터가 들어오지 않을 때 Codex 가 관리자 PowerShell 에서 아래 규칙을 점검/추가할 수 있다. 동일 이름 규칙이 이미 있으면 중복 생성하지 않도록 한다.

```
$RuleName = "ESP32 IMU UDP 5005"
$existing = Get-NetFirewallRule -DisplayName $RuleName -ErrorAction SilentlyContinue
if (-not $existing) {
    New-NetFirewallRule `
        -DisplayName $RuleName `
```

```

-Direction Inbound `
-Action Allow `
-Protocol UDP `
-LocalPort 5005
Write-Host "Firewall rule created: $RuleName"
} else {
Write-Host "Firewall rule already exists: $RuleName"
}

```

7. Codex 작업 후 실제 실행 순서

단계	확인 내용
A. 노트북	Dofbot 핫스팟에 연결하고 VSCode 또는 PowerShell 에서 receiver 를 실행한다.
B. ESP32 펌웨어	소스 수정이 있으면 USB 연결 상태에서 COM3 로 115200 flash 한다.
C. USB 제거	flash 완료 후 ESP32 를 별도 전원으로 켜고 USB 데이터 케이블을 제거한다.
D. 무선 수신	노트북에서 imu_receiver.py 가 5005 포트를 listen 하고 있는지 확인한다.
E. 장갑 테스트	장갑을 전/후/좌/우/대각선으로 움직여 STATE 와 각도 값이 계속 변하는지 확인한다.

```

# ESP32 빌드
cd C:\Users\kdg57\esp\esp32_test
idf.py build

# 필요 시 플래시
idf.py -p COM3 -b 115200 flash

# 노트북 무선 receiver
python C:\Users\kdg57\imu_receiver\imu_receiver.py

```

8. 실패 시 Codex 가 확인할 우선순위

증상	우선 확인
Receiver 가 Waiting 만 표시	노트북과 ESP32 가 같은 Dofbot 에 연결됐는지 확인 -> Windows 방화벽 UDP 5005 확인 -> broadcast 대신 노트북 IPv4 unicast 적용.
ESP32 Wi-Fi 연결 실패	SSID/비밀번호 확인, 휴대폰 핫스팟 2.4 GHz 사용 가능 여부 확인, ESP32 로그의 IP_EVENT_STA_GOT_IP 확인.
UDP 는 오지만 값이 이상함	네트워크 코드는 건드리지 말고 기존 MPU6050 계산/축 부호/임계값 로직과 이전 정상 버전을 비교.
idf.py build 실패	main/CMakeLists.txt 가 esp32_test.c 를 가리키는지 확인하고 실제 파일 존재 여부 확인.
flash Serial noise/corruption	센서/외부 GPIO 부하를 일시 분리하고 115200 baud 로 flash. flash 후 센서 재연결.
COM8/COM9 혼동	COM8/COM9 는 Bluetooth serial 이고 ESP32 는 CP210x COM3 기준.

9. 최종 성공 판정 예시

USB 케이블을 제거한 상태에서 ESP32 는 별도 전원으로 동작하고, Windows receiver 에서 아래처럼 연속 데이터가 들어오면 1 차 무선 IMU 통신 성공이다.

ESP32 MPU6050 Wi-Fi Receiver

Listening UDP port : 5005

Waiting for ESP32...

[21:35:11.125] [ESP32 192.168.43.25] ROLL: 0.82 | CTRL_ROLL: -0.82 | PITCH: -1.12 | STATE:STOP

[21:35:11.176] [ESP32 192.168.43.25] ROLL: 3.22 | CTRL_ROLL: -3.22 | PITCH:-19.55 | STATE:FORWARD

[21:35:11.228] [ESP32 192.168.43.25] ROLL: 21.45 | CTRL_ROLL: -21.45 | PITCH: -22.80 | STATE:FORWARD_LEFT

이 단계가 성공한 다음에만 imu_receiver.py 를 ROS2 노드로 변경하여 STATE 를 geometry_msgs/Twist 로 변환하고 /cmd_vel 로 발행하는 단계로 확장한다.